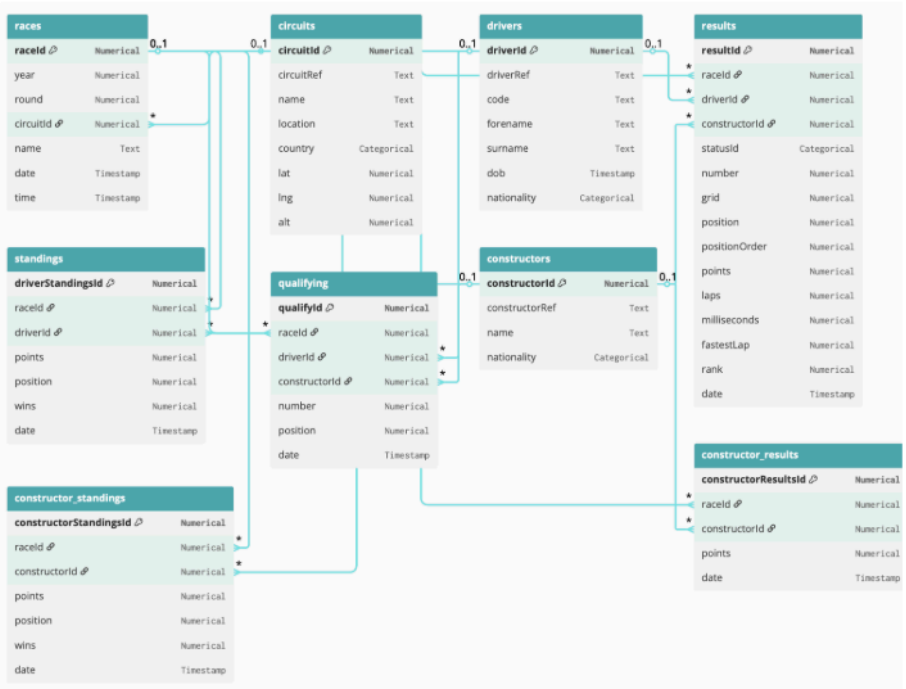
# Predicting Driver Finishing Position with Relational Deep Learning

*RelBench rel-f1 dataset*

## Portfolio Project Report





# Maya Tem

Master's in Data Science

April 2026

**Introduction** **2**
**Background** **2**
**Dataset and Prediction Task** **2**
**Methods** **4**
Overall Workflow 4
Baseline Models 4
Graph Construction and Graph Neural Network 5
Training and Evaluation Procedure 5
**Results** **6**
**Discussion** **8**
**Limitations** **8**
**Future Work** **9**
**Conclusion** **9**
**References** **11**
**Appendix** **11**

## Introduction

In this project, I studied how relational deep learning can be used on a realistic multi-table dataset. Instead of working with a single flat table, I used the RelBench `rel-f1` benchmark, which is based on Formula 1 data stored across connected relational tables. The database contains 9 tables, 97,606 rows, and 77 columns, and it tracks Formula 1 data since 1950.

My main goal was to compare simpler flat-feature models with a graph-based relational deep learning model. I wanted to understand whether keeping the relational structure of the data would improve prediction performance compared with flattening the data into a smaller set of engineered features. To study this, I used the `driver-position` task, which asks the model to predict the average finishing position of each driver over future races and is evaluated with mean absolute error (MAE).

I also used this project as a learning opportunity. In addition to training and evaluating multiple models, I wanted to better understand how relational databases, graph representations, and deep learning can be connected in a practical data science workflow.

## Background

In many real-world applications, data is not stored in a single flat table. Instead, it is stored across multiple related tables connected by primary keys and foreign keys. This is also true for the `rel-f1` dataset, which includes connected information about `drivers, constructors, races, standings, qualifying records,` and `results.`

In a traditional machine learning workflow, I would usually join or flatten these tables into one tabular dataset and then build a model from engineered features. This approach can still be useful, but it may lose some of the information contained in the original relational structure. For this reason, I wanted to explore relational deep learning as a way to keep these connections and use them more directly during modeling.

For this project, RelBench gave me a structured way to study this problem. It provided a benchmark task, predefined data splits, and a clear framework for comparing flat baseline models with a graph-based relational deep learning model.

## Dataset and Prediction Task

In this project, I used the RelBench `rel-f1` dataset, which is based on Formula 1 data stored across multiple connected tables. Instead of representing the data as a single flat table, the dataset keeps the original relational structure. This made it a good choice for studying how relational deep learning can use both feature information and table relationships.

The `rel-f1` database includes several important tables, such as `drivers, races, results, constructors, standings, circuits`, and `qualifying`. These tables are connected through primary key and foreign key relationships. Because of this structure, the dataset reflects a more realistic database setting than a standard flat machine learning dataset.

The `rel-f1` dataset was especially suitable for my project because it allowed me to compare two different modeling approaches. On one side, I built flat baseline models by creating engineered features from the relational data. On the other side, I preserved the relational structure and converted the database into a graph for relational deep learning.

Database schema:

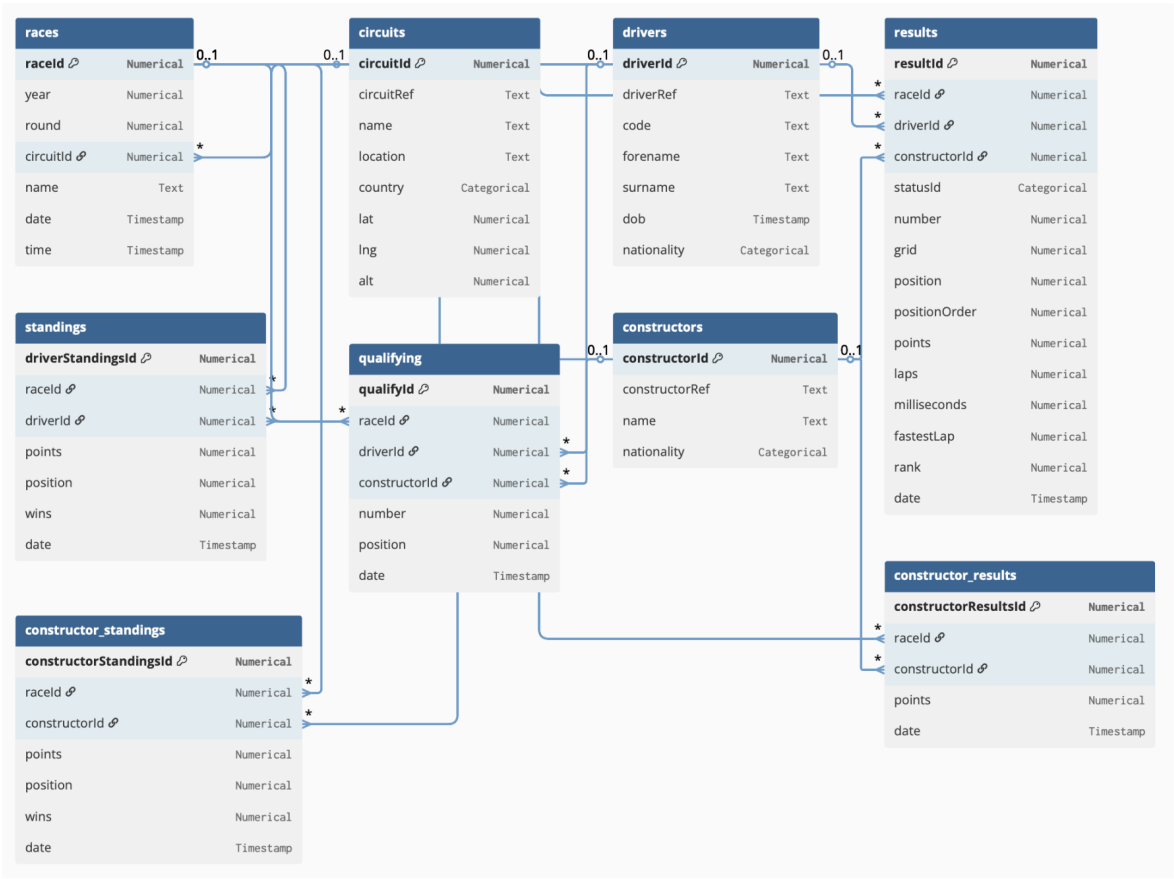



***Figure 1.*** *Schema of the RelBench `rel-f1` database, showing the main tables and the primary key / foreign key relationships among races, drivers, constructors, results, standings, circuits, and qualifying records.* [https://relbench.stanford.edu/datasets/rel-f1/](https://relbench.stanford.edu/datasets/rel-f1/)

For the prediction task, I selected `driver-position`. This is a regression task in which the goal is to predict a driver's average finishing position over a future time window. I selected this task because it allowed me to compare baseline models and a graph neural network on the same problem while keeping the project manageable in Google Colab.

Since the target variable is continuous, I treated this as a regression problem. I mainly used mean absolute error (**MAE**) to evaluate model performance. I also reported root mean squared error **(RMSE)** and the coefficient of determination **($R^2$)** to give a broader view of model quality.

## Methods

### Overall Workflow

In this project, I compared flat-feature baseline models with a graph-based relational deep learning model on the same prediction task. I first loaded the `rel-f1` dataset and selected the `driver-position` task from RelBench. Then I used the predefined training, validation, and test splits provided by the benchmark.

My overall workflow had two main parts:

- First, I built flat baseline models by creating simple historical driver-level features from the training data.
- Second, I converted the relational database into a heterogeneous graph and trained a graph neural network on that graph representation. This allowed me to compare standard machine learning, standard deep learning, and relational deep learning within the same project.

## Baseline Models

Before training the graph neural network, I built three baseline models for comparison:

- a Mean Baseline,
- a Random Forest Regressor, and
- an MLP baseline.

**For the flat-feature setting**, I created simple driver-level historical features from the training split. These features included the driver's mean finishing position, standard deviation of finishing position, minimum finishing position, maximum finishing position, and number of observed races in the training data. I then merged these features into the training and validation tables by `driverId`.

| | driverId | driver_mean_position | driver_std_position | driver_min_position | driver_max_position | driver_num_races |
|---|---|---|---|---|---|---|
| **0** | 1 | 12.504000 | 3.427555 | 6.250000 | 19.50 | 25 |
| **1** | 3 | 10.074444 | 4.728703 | 3.000000 | 17.25 | 15 |
| **2** | 7 | 10.629167 | 4.922590 | 2.666667 | 21.50 | 20 |
| **3** | 9 | 11.000000 | 5.656854 | 7.000000 | 15.00 | 2 |
| **4** | 10 | 10.681818 | 3.254232 | 5.000000 | 15.50 | 11 |

***Table 1.*** *Example of the engineered flat features created for the baseline models, including driver mean position, standard deviation, minimum position, maximum position, and number of observed races.*

The Mean Baseline predicted a constant value equal to the average target value in the training set. I used this model as a simple reference point.

**The Random Forest Regressor** served as the classical machine learning baseline. I used it to test how well a non-deep-learning model could perform when given manually engineered flat features.

**The MLP baseline** served as the standard deep learning model in the flat-feature setting. I trained it on the same engineered features used by the Random Forest model. This made it possible to compare classical machine learning, standard deep learning, and relational deep learning more clearly.

| | Model | Validation MAE | Validation RMSE | Validation R2 |
|---|---|---|---|---|
| 0 | Mean Baseline | 4.334385 | 5.425462 | -0.369534 |
| 1 | Random Forest | 3.706259 | 4.593106 | 0.018450 |
| 2 | MLP | 3.755330 | 4.640538 | -0.001927 |

***Table 2*** *Validation results of the flat-feature baseline models, showing that the Random Forest model performed slightly better than the MLP baseline.*

## Graph Construction and Graph Neural Network

To build the relational deep learning pipeline, I converted the relational database into a heterogeneous graph by using the primary key and foreign key relationships defined in the dataset. In this graph, each table became a node type, and the connections between tables became edge types.

During graph construction, I simplified text-embedded columns into categorical columns. I made this decision because I encountered compatibility issues when trying to use external text embedding components in Google Colab. This simplification allowed me to keep the pipeline stable and complete the project with a working end-to-end graph representation.

After constructing the graph, I prepared mini-batch neighborhood loaders for the training, validation, and test splits. I used neighborhood sampling because the full graph was too large to process at once. This made graph training feasible within the available Colab resources.

For the main model, I used a heterogeneous graph neural network based on the RelBench tutorial design. The model included a feature encoder, a temporal encoder, GraphSAGE message passing layers, and an MLP prediction head. I used this model because it directly learned from the relational structure of the database instead of relying only on manually engineered flat features.

## Training and Evaluation Procedure

I trained all models by using the predefined training split and used the validation split for model selection. For the baseline models, I evaluated performance on the validation set after fitting each model.

For the graph neural network, I used mini-batch training with neighborhood sampling. I trained the model for multiple epochs, evaluated it on the validation set after each epoch, and kept the model state that produced the best validation MAE. Since the task was a regression problem, I used L1 loss during GNN training.

To compare model performance, I mainly used MAE. I also reported RMSE and $R^2$ to give a broader view of model quality. After selecting the best GNN model on the validation set, I reloaded the best saved model state and generated final predictions for the test split.

```
Best validation metrics: {'r2': 0.23547378755497583, 'mae': 3.2502113969149233, 'rmse': 4.053652161387901}
```

***Figure 2.*** *Best validation performance of the graph neural network model, reported in terms of MAE, RMSE, and $R^2$ after model selection on the validation split.*

## Results

I compared four models on the `driver-position` regression task: a Mean Baseline, a Random Forest Regressor, an MLP baseline, and a graph neural network based on the RelBench tutorial design.

The validation results showed clear differences across the models. The Mean Baseline produced the weakest performance, with a validation MAE of **4.3344**, an RMSE of **5.4255**, and an $R^2$ of **-0.3695**. This was expected, since the model only predicted a constant value and did not learn any useful structure from the data.

The Random Forest model improved substantially over the Mean Baseline. It achieved a validation MAE of **3.7063**, an RMSE of **4.5931**, and an $R^2$ of **0.0185**. This result showed that even simple historical driver-level features contained useful predictive information.

The MLP baseline also performed better than the Mean Baseline, with a validation MAE of **3.7553**, an RMSE of **4.6405**, and an $R^2$ of **-0.0019**. However, it performed slightly worse than the Random Forest model on all main validation metrics. This suggested that, in the flat-feature setting used in this project, the standard deep learning baseline did not outperform the classical machine learning baseline.

The graph neural network achieved the best overall validation performance among all models. It produced a validation MAE of **3.2502**, an RMSE of **4.0537**, and an $R^2$ of **0.2355**. Compared with the Random Forest baseline, the GNN reduced validation MAE by about **0.4561** and improved $R^2$ by a noticeable margin. This indicated that preserving the relational structure of the database helped the model capture predictive information beyond what was available in the flat engineered features.

A summary of the main validation results is shown below.

| | Model | Validation MAE | Validation RMSE | Validation R2 |
|---|---|---|---|---|
| 0 | Mean Baseline | 4.334385 | 5.425462 | -0.369534 |
| 1 | Random Forest | 3.706259 | 4.593106 | 0.018450 |
| 2 | MLP | 3.755330 | 4.640538 | -0.001927 |
| 3 | GNN | 3.250211 | 4.053652 | 0.235474 |

***Table 3.*** *Validation performance comparison across the Mean Baseline, Random Forest, MLP, and GNN models.*

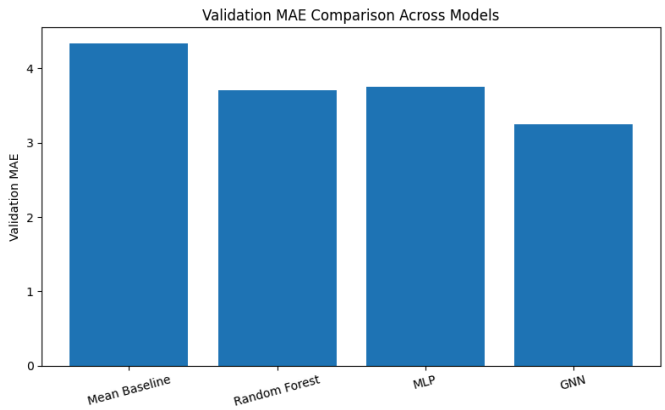



***Figure 3.*** *Validation MAE comparison across models. Lower MAE indicates better prediction performance. The GNN achieved the lowest MAE among all models.*

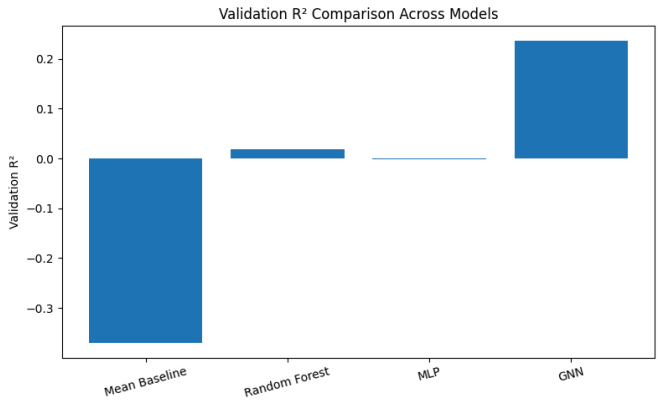



***Figure 4.*** *Validation $R^2$ comparison across models. Higher $R^2$ indicates a better fit to the target values. The GNN achieved the highest $R^2$ among all models.*

As shown in Table 3 and Figures 3–4, the graph neural network achieved the strongest overall validation performance. It produced the lowest MAE and the highest $R^2$ among all models. The Random Forest baseline performed better than the MLP in the flat-feature setting, but both were outperformed by the graph-based model.

## Discussion

In this project, I observed that the graph neural network was the strongest model among all the approaches I tested. It outperformed the Mean Baseline, the Random Forest model, and the MLP baseline on all main validation metrics. This result suggests that preserving the relational structure of the data provided useful predictive information that was not fully captured by flat engineered features alone.

I also observed that the Random Forest model performed better than the MLP in the flat-feature setting. This is an important result because it shows that a standard deep learning model does not automatically outperform a classical machine learning model when the feature set is relatively simple. In this project, the manually engineered historical driver-level features were useful, but they were still limited compared with the full relational structure available in the original database.

At the same time, I do not interpret this project as a strict apple-to-apple comparison between identical models. The baseline models and the graph neural network were built on different data representations. The flat models used engineered tabular features, while the graph neural network used the heterogeneous relational graph directly. For this reason, I see this project as a comparison of modeling pipelines rather than a controlled ablation study. Even so, I believe the comparison is still meaningful because all models were tested on the same prediction task, with the same benchmark splits and the same evaluation metrics.

From a practical data science perspective, this result is important because real-world data is often stored in relational databases rather than in a single flat table. In many applied settings, a large amount of effort goes into joining, aggregating, and flattening data before modeling. This project helped me see that relational deep learning can offer a different approach by learning directly from the structure of connected tables instead of relying only on manual feature engineering.

Overall, I found that this project supported the main idea I wanted to test: using the relational structure of the database can improve predictive performance. At the same time, the baseline models were still valuable because they showed that simple historical features already contain useful signals. This made the final comparison more informative and helped me better understand when relational deep learning may provide an advantage.

## Limitations

This project has several limitations. To keep the experiment reproducible in a standard Google Colab environment, I used the smaller RelBench `rel-f1` dataset rather than a larger benchmark such as `rel-amazon`. This made the project more feasible, but it also limited the scale of the experiments.

Second, I simplified the handling of text-based columns during graph construction. Instead of using a full external text embedding pipeline, I treated text-embedded columns in a simpler way to avoid compatibility problems and to keep the pipeline stable. As a result, the graph

representation used in this project was practical and workable, but it did not fully use all possible information from the text fields.

Third, the graph neural network model was based on the RelBench tutorial design rather than a completely new custom architecture. My main contribution was not proposing a new graph model, but building a complete experimental pipeline and comparing relational deep learning with multiple baseline models.

Another limitation is that hyperparameter tuning was limited. Because of time and compute constraints, I used a practical number of epochs and a small set of design choices instead of performing a more extensive hyperparameter search.

Finally, the main model comparison in this report was based on validation performance. Although I generated final predictions for the test split, the main comparative interpretation focused on the validation results.

## Future Work

There are several ways this project could be extended in the future. One possible improvement would be to apply the same pipeline to a larger RelBench dataset if more memory and compute resources are available. This would make it possible to test whether the same conclusions hold in a larger and more complex relational setting.

Another extension would be to improve the handling of text-based columns. In a future version, I could integrate a more complete text embedding pipeline and study whether richer textual information improves graph-based prediction performance.

It would also be useful to explore additional hyperparameter tuning for both the baseline models and the graph neural network. For example, future experiments could test different hidden dimensions, neighborhood sampling sizes, numbers of layers, learning rates, and training durations.

Finally, future work could include trying alternative relational deep learning architectures and comparing them with the current GraphSAGE-based model. This would help determine whether the performance gain comes mainly from the relational graph representation itself or also from the specific model design.

## Conclusion

In this project, I explored how relational deep learning can be applied to a realistic multi-table dataset by using the RelBench `rel-f1` benchmark. My main goal was not only to train a graph neural network, but also to understand how relational data can be modeled in different ways and to compare simpler baselines with a relational deep learning approach.

To do this, I compared four models: a Mean Baseline, a Random Forest Regressor, an MLP baseline, and a graph neural network based on the RelBench tutorial design. This comparison allowed me to evaluate the problem from multiple perspectives, including naive prediction,

classical machine learning, standard deep learning on flat features, and relational deep learning on graph-structured data.

The results showed that the graph neural network achieved the best validation performance among all models. This suggests that representing the database as a heterogeneous graph provided useful information that could not be fully captured by flat feature engineering alone. In other words, the relational structure itself contributed to better prediction.

From a data science perspective, this project was especially valuable because most real-world data is stored in relational databases rather than in a single flat table. In practice, data scientists often spend a large amount of time preparing, joining, and flattening data before modeling. By using RelBench, I was able to work directly with a relational benchmark and focus on how connected tables can be used for machine learning and deep learning in a more structured way.

I also found that RelBench was a very useful framework for learning and experimenting with relational deep learning. It provided a realistic dataset, predefined tasks, and a benchmark structure that made it easier to study this area without spending excessive time on raw dataset collection and manual benchmark design.

Overall, this project helped me better understand the difference between flat-feature modeling and relational deep learning. It also gave me practical experience in building, comparing, and evaluating multiple models in a structured way. For me, this project was not only about obtaining a final result, but also about learning how relational databases and deep learning can be connected in a meaningful data science workflow.

## References

Robinson, J., Ranjan, R., Hu, W., Huang, K., Han, J., Dobles, A., Fey, M., Lenssen, J. E., Yuan, Y., Zhang, Z., He, X., & Leskovec, J. (2024). *RelBench: A benchmark for deep learning on relational databases*. arXiv. https://arxiv.org/abs/2407.20060

RelBench. (n.d.). *rel-f1 dataset*. Stanford University. https://relbench.stanford.edu/datasets/rel-f1/

RelBench. (n.d.). *RelBench documentation*. Stanford University. https://relbench.stanford.edu/

**Temporal Graph Learning. (2024, Nov. 21).** *RELBENCH: A benchmark for deep learning on relational databases* [Video]. YouTube. https://www.youtube.com/watch?v=2idUm4OBX7Y

## Appendix

The full implementation is available in the accompanying Google Colab notebook. The notebook includes data loading, graph construction, baseline model training, GNN training, validation evaluation, and final test prediction generation.